Thaghrawy — Architecture & Folder Breakdown

An AI-powered penetration-testing assistant with persistent cross-engagement memory.

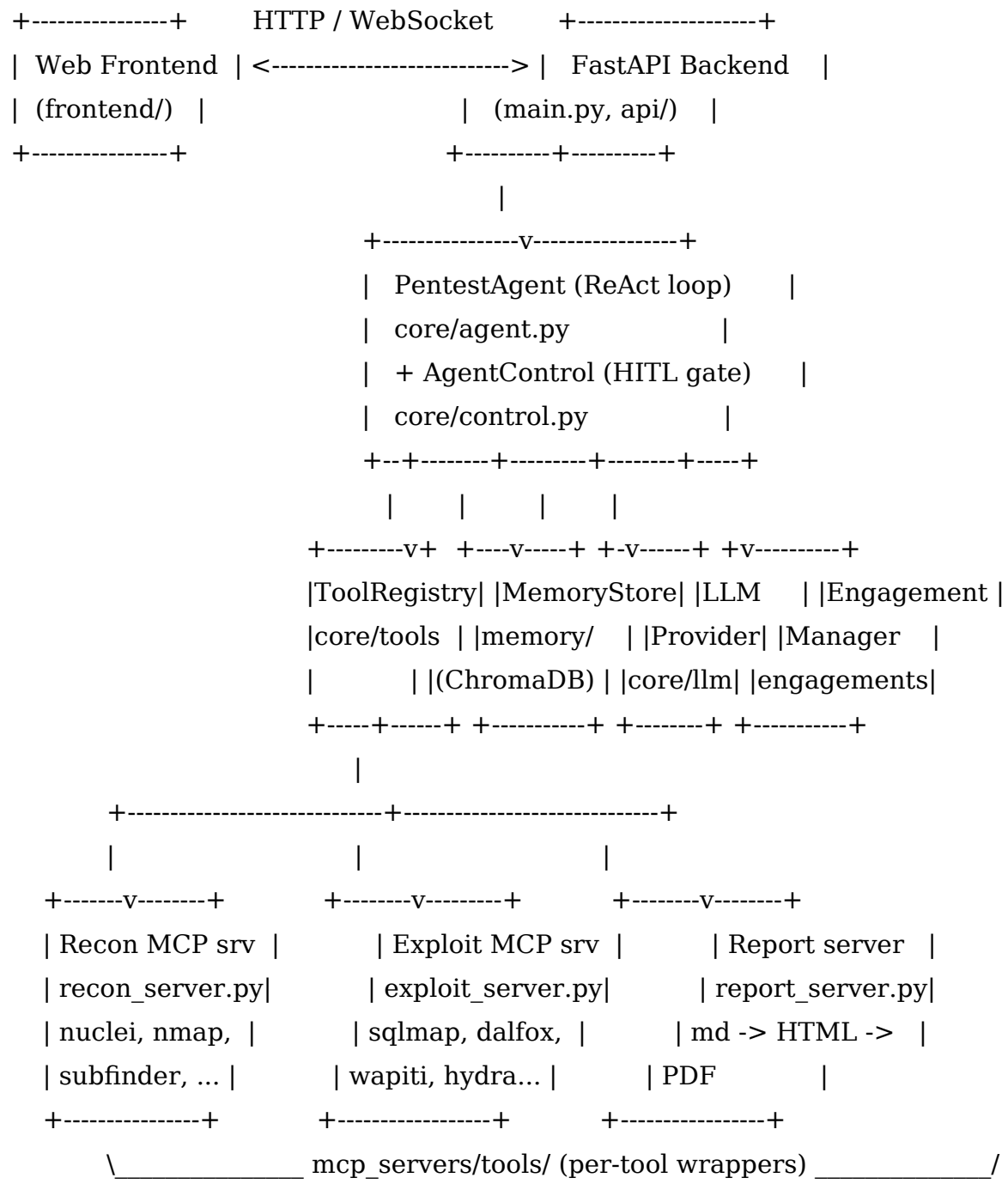
1. Overview

Thaghrawy is a Python/FastAPI application that lets a Large Language Model (LLM) drive a suite of ~29 real security tools through a standard interface (the Model Context Protocol, MCP). A ReAct-style agent reconnoitres a target, interprets tool output, saves findings to a ChromaDB semantic-memory store, and produces two reports (a technical CVSS report and an executive DREAD report). Engagements run in phases — autonomous enumeration, a human-in-the-loop (HITL) collaboration phase where every tool call can be approved / rejected / edited, and reporting.

Tech stack: Python 3 + FastAPI (REST + WebSocket), a custom ReAct agent, MCP tool servers (subprocess wrappers around the CLIs), ChromaDB + `sentence-transformers` for memory, a multi-provider LLM layer (Anthropic / OpenAI-compatible / Ollama, local LM Studio by default), and a Markdown → HTML → PDF reporting pipeline.

How it runs: Docker-first — `docker compose up --build` starts just the agent (all tools baked into the image); `docker compose --profile targets up` adds the optional DVWA and Juice Shop practice targets. See `README.md` for full details.

2. High-Level Architecture



The backend hosts a per-engagement `PentestAgent`. The agent is wired to a `ToolRegistry` (all tools), a `MemoryStore` (ChromaDB), an `LLMProvider`, an `AgentControl` (the HITL channel), and an `EngagementManager`. Tools are grouped into recon and exploit MCP servers; a report server renders findings to PDF.

3. Request / Data Flow

A. Interactive chat turn (POST-less, over /ws/chat):

1. api/websocket.py receives a chat frame and calls PentestAgent.chat().
2. The agent searches memory (MemoryStore.search_context) and builds a system prompt (prompt_builder.py, injecting memory hits + methodology from skills.py).
3. The ReAct loop streams the LLM's tokens and tool calls. For each tool call, if a control channel is active in the collaboration phase, the agent **pauses** and emits tool_call_pending; the human approves / rejects / edits (core/control.py).
4. Approved calls run through ToolRegistry.execute(); the result is fed back into the loop. Findings the human saves are persisted to ChromaDB.
5. Every decision is appended to the engagement's trajectory log for training data.

B. Autonomous enumeration → handoff:

1. PentestAgent.enumerate() runs deterministic recon probes (e.g. nuclei_scan).
2. core/finding_drafts.py::finding_from_tool_result turns structured scanner output into Finding objects (with vuln_type normalized to the benchmark vocabulary) and auto-ingests them — no LLM, no approvals.
3. The engagement phase flips to collaboration; the agent emits a handoff so the human takes over with the approval gate on.

4. Folder-by-Folder Breakdown

core/ — the agent brain

The orchestration layer: the agent loop, LLM abstraction, tool registry, and HITL control.

File	Purpose
agent.py	PentestAgent — the tool-calling ReAct loop (chat(), enumerate()), streams normalized events, applies the approval gate.
llm.py	Provider abstraction (BaseLLMProvider.stream()) for Anthropic / OpenAI-compatible / Ollama; normalized streaming + tool-use events.
tools.py	The unified ToolRegistry — **all** agent tools are defined/wired here; build_default_registry / build_filtered_registry (recon-only vs full).
context.py	Context-window trimming and tool-output truncation so long scans don't blow the token budget.
control.py	AgentControl — the HITL channel (asyncio queue + phase / auto-approve / stop state) that gates tool calls.
finding_drafts.py	Turns raw scanner output (nuclei/sqlmap/dalfox/wapiti/nikto) into Finding drafts for auto-ingest and human promotion.

memory/ — persistent semantic memory

Cross-engagement recall of findings and techniques.

File	Purpose
store.py	MemoryStore — the ChromaDB interface. **All memory operations go through here** (add/search/update/delete findings & techniques).
embeddings.py	Local sentence-transformers wrapper (all-MiniLM-L6-v2) that embeds text for semantic search.
schemas.py	Pydantic models: Finding, Technique, Engagement (incl. cvss_score, dread_score).

engagements/ — engagement lifecycle

Path	Purpose
manager.py	EngagementManager — create/update engagements, phase & step tracking, JSON + Markdown session logs.
sessions/	Per-engagement data at runtime: <id>.json, Markdown logs, and <id>.trajectory.jsonl (HITL decisions for training).

mcp_servers/ — the tools, as MCP servers

Wraps the security-tool CLIs as MCP tools the agent can call. Named mcp_servers/ (not mcp/) to avoid shadowing the installed mcp SDK package.

Path	Purpose
recon_server.py	Registers the recon/scan tools (subdomain enum, port scan, HTTP probe, fuzz, crawl, vuln templates, TLS/WAF, param discovery, etc.).

Path	Purpose
exploit_server.py	The actively-attacking tools (sqlmap, nikto, hydra, dalfox, wapiti…), registered dangerous=True.
report_server.py	Renders Markdown → .md/.pdf (render_to_files()); no memory/engagement dependency.
tools/	~26 per-tool wrapper modules (nmap.py, nuclei.py, sqlmap.py, dalfox.py, wapiti.py, httpx.py, ffuf.py, …).
tools/_common.py	Shared helper: run_command() (timeout + workspace persistence + JSON envelope), strip_url(), resolve_host(), sanitize_input().

*Note: the wrappers return **JSON strings**, so consumers json.loads() before use.*

api/ — HTTP & WebSocket surface

Path	Purpose
deps.py	Dependency wiring: builds the per-engagement agent + control channel (reads analysis_mode to pick the tool registry).
websocket.py	The chat WebSocket: concurrent reader/worker, slash-command parsing, streams agent events.
routes/chat.py	Chat REST endpoints.
routes/engagements.py	Engagement CRUD + phase/mode control.
routes/findings.py	Findings curation: PATCH (edit), DELETE (mark FP), promote result→draft.
routes/memory.py	Memory search endpoints.

Path	Purpose
routes/reports.py	Generate / list / download reports.
routes/tools.py	Human-run-a-tool endpoint.
routes/training.py	Export training data (SFT / DPO).
routes/lm_studio.py	Local LLM (LM Studio) status/health checks.

reporting/ — report content

File	Purpose
builder.py	**Pure** functions that turn an Engagement + its Findings into the technical and executive report Markdown (no I/O, no DB).

benchmarks/ — evaluation harness

Scores an engagement against the project's four metrics.

File	Purpose
scorer.py	**Pure** scoring: ESR, AST, FP-rate, Detection Rate → a BenchmarkResult.
runner.py	I/O driver (python -m benchmarks.runner <id> <target>).
ground_truth.py	Known DVWA/Juice-Shop vulnerability categories used to score detection.
README.md	Metric definitions and usage.

training/ — fine-tuning data export

File	Purpose
exporter.py	Turns findings/techniques into SFT examples and HITL trajectories into DPO preference

File	Purpose
	pairs.
README.md	Export formats and usage.

frontend/ — the UI

File	Purpose
index.html	The single-page, three-panel layout (engagements / chat / findings).
app.js	WebSocket client, streaming render, approve/reject/edit controls, slash commands.
style.css	The dark hacker-themed styling.

tests/ — the 493-test suite

Mirrors the source layout — test_core_agent.py, test_memory_store.py, test_control.py, test_websocket.py, test_benchmarks*.py, test_exporter.py, the API route tests, etc. tests/mcp_tools/ unit-tests each tool wrapper (with mocked subprocess). conftest.py holds shared fixtures.

scripts/ — operational scripts

File	Purpose
tool_smoke.py	Live smoke harness — drives **every** registered tool through ToolRegistry.execute() against an owned target; catches real-CLI bugs mocks miss.
run_pentest.py	End-to-end driver: real agent → findings → reports → benchmark scoring.
export_training_data.py	CLI for the training exporter (--format messages|sft|preference).

docs/ — documentation

Path	Purpose
graduation/	The graduation deliverables (comparison report, Final Documentation, Appendix A) + reference.docx styling template.
architecture/	This document.

Runtime / data & CI directories

Path	Purpose
reports/	Generated technical/executive reports (may contain sensitive data).
chroma_db/	ChromaDB vector store (Docker named volume chroma_data; survives rebuilds).
engagements/sessions/	Per-engagement JSON + trajectory logs (Docker volume engagement_data).
.github/	CI workflow (ci.yml: .env check → flake8 → pytest).

5. Root-Level Files

File	Purpose
main.py	FastAPI app entry point — serves the frontend at /, REST at /api/, chat at /ws/chat.
config.py	Central configuration from env vars (.env) — LLM provider, model, paths, limits.
guardrails.py	Safety layer: dangerous-shell-pattern gating + JSON enforcement + command logging. **Never bypassed.**
prompt_builder.py	Builds the system prompt; injects memory hits and methodology into every turn.
skills.py	Methodology guidance — maps each pentest phase to relevant tools; injected via prompt_builder.py.
output_filter.py	Per-tool output truncation/extraction (e.g. nmap/sqlmap/nikto).
requirements.txt	Python dependencies.
Dockerfile	Builds the image with all ~29 tools; modular torch backend (COMPUTE_BACKEND=cpu|cuda|rocm).
docker-compose.yml	Primary run config — the agent service; DVWA/Juice Shop behind a targets profile.
docker-compose.gpu-nvidia.yml / docker-compose.gpu-amd.yml	GPU overlays (CUDA / ROCm).
.mcp.json	MCP server declarations.
.env.example	Template for the required environment

File	Purpose
	configuration.
CONTRIBUTING.md	Developer & architecture guide (code rules, how to add tools/providers, workflow).
README.md	User-facing quickstart, Docker run guide, and architecture summary.

6. How the Layers Depend on Each Other (Rules of the Road)

These conventions keep the system consistent — they mirror CONTRIBUTING.md:

- **All tool calls go through `core/tools.py`** (the ToolRegistry). The agent never invokes a CLI directly.
- **All memory operations go through `memory/store.py`.** Never query ChromaDB directly.
- **MCP tool wrappers return JSON strings**, produced via `mcp_servers/tools/_common.py`'s `run_command()`. Consumers `json.loads()` the result.
- **`guardrails.py` is never bypassed** — it exists for legal/scope compliance, and the `shell` tool logs every command.
- **Type hints on every function; Pydantic models for all data structures.**
- **Analysis mode gates capability at the registry level:** a `recon_only` engagement never even registers the exploit tools, so the model physically cannot attempt exploitation.
- **Reporting and scoring are pure:** `reporting/builder.py` and `benchmarks/scorer.py` do no I/O, which makes them easy to test and reuse.

Directional dependency summary

```
frontend/ -> api/ -> core/agent.py -> core/tools.py -> mcp_servers/ -> (tool CLIs)
                          |              |
                          v              v
                   memory/store.py    core/control.py (HITL)
                          |
                          v
                   ChromaDB (chroma_db/)

engagements/manager.py  <->  core/agent.py        (lifecycle, phases, steps, trajectory)
```

reporting/builder.py -> mcp_servers/report_server.py (Finding -> Markdown -> PDF)
benchmarks/scorer.py <- engagements + memory (evaluate a finished engagement)
training/exporter.py <- findings + trajectory logs (SFT / DPO datasets)